⚙️ Learn Back-End Web Development

Fulllion Creative Works

Versions and facts checked in November 2026. They change, so confirm them on each project's site.

Contents

1. What the Back End Does
2. The Five Steps
3. The Minimum to Learn First
4. Node.js and Express: A Common Alternative
5. Languages to Learn Later
6. Free Courses and Roadmaps
7. HTTP, REST APIs and JSON
8. Databases: SQL First, NoSQL When It Fits
9. Authentication Basics
10. Web Security and the OWASP Top 10
11. Environment Variables and Secrets
12. Testing
13. Deployment Basics
14. Tools: Charm, Postman, curl and Docker
15. Common Mistakes
16. Sources

---

1. What the Back End Does

The **front end** is what runs in the user's browser: the page, the buttons, the layout. The **back end** runs on a server you control. When the front end needs something it can't do by itself, it sends the back end a request.

```
Browser or app ──"show me my notes"──────────────▶ Your server
Your server    ──checks who's asking──────────────▶ (sign-in)
Your server    ──reads the notes──────────────────▶ Database
Your server    ──sends the notes back as JSON─────▶ Browser or app
```

A back end usually:

1. **Receives requests** over HTTP, the language of the web (Section 7).
2. **Checks who's asking**, and whether they're allowed to do it (Section 9).
3. **Reads and writes data** in a database (Section 8).

4. **Sends back a response**, usually JSON for apps, or a finished HTML page.

The back end is where you can keep secrets and enforce rules, because users can't see or change its code. Anything in the front end, users can read and change.

New to how the pieces fit? Learn Web Development: Start Here covers web app architecture, including ForrestKnight's video on it.

---

2. The Five Steps

The same five steps work for the back end as for the front end:

1. **Learn a language to a basic proficiency.**
2. **Build basic applications and projects.**
3. **Learn the libraries, frameworks and runtime environments** related to that language.
4. **Build more complex projects** that use frameworks and more than one language. By now you should be, or become, familiar with **Git and GitHub**, and create a repository for each project if you haven't already.
5. **Create a portfolio website** that shows your work, and pursue your career.

You can learn the topics below in any order. Just don't take on too many at once: one language and one framework, used to build real things, beats five half-learned ones. Learning How to Learn Programming has tips for studying, and for getting out of tutorial hell.

For step 2, Coding Practice and Interview Prep lists challenge and practice-project sites. Back-end work also leans on the ideas in Computer Science Fundamentals: data structures, algorithms and Big O explain why one database query or loop is fast and another crawls, and design patterns show up all over back-end frameworks.

---

3. The Minimum to Learn First

For a quick checklist of the topics back-end developers rely on, watch Aaron Jack's short, Learning Back End? You NEED to know these topics. His point is that the back end asks for more than basic SQL, and his list maps onto this guide:

His topic	What it means	Where it's covered
Databases beyond queries: backups, scaling, ORMs, migrations, SQL versus NoSQL	Keeping data safe and fast as an app grows	Sections 8 and 13
ACID	A database transaction is all or nothing, leaves the data valid, doesn't trip over other transactions, and survives a crash once it's saved	Section 8

His topic	What it means	Where it's covered
The CAP theorem	When the network between parts of a distributed database fails, it has to choose between giving consistent answers and staying available	Background for when you outgrow one database
Security and encryption	HTTPS, password hashing, and keeping secrets secret	Sections 9 to 11
Kinds of APIs, and API design	REST first; GraphQL and others later	Section 7
Observability: monitoring and logging	Knowing what your server is doing, and why it failed	Section 13, and Getting Started with DevOps

He ends on an encouraging note: front-end work being easier to start with is a good thing for beginners, not a reason to look down on it. You don't need all of this before your first back-end project.

The bare minimum to begin job hunting as a back-end developer:

Learn	Why	Current version (November 2026)
Python	Readable, beginner-friendly, and used everywhere from web servers to data work and AI	**3.15** (3.15.2). 3.14 also still gets bug fixes
Java	A mainstay of large companies, banks and Android back ends, usually with the Spring Boot framework	**Java 25** is the current long-term support (LTS) release. Java 27 is newer, but not LTS
SQL	The language of relational databases, which almost every back end uses	PostgreSQL **18** (18.6). SQLite is built into Python
Flask and/or Django	Python's two best-known web frameworks	**Django 6.1** (6.1.3), or **5.2 LTS**. **Flask 3.1** (3.1.3)
Git and GitHub	Version control, and where employers look at your code	See the CLI Cheat Sheet's Git section

A few notes on choosing:

- **Learn Java on an LTS release.** LTS releases get updates for years, and most jobs use them. Oracle's next planned LTS is Java 29.
- **Flask or Django? Flask** is small: you add pieces as you need them, which makes it easy to see how things work. **Django** comes with almost everything built in: a database layer (the ORM), sign-in, an admin site and security protections. Start with Flask to learn the ideas, or Django to build a complete site quickly. Both are worth knowing.
- **Django and Python versions go together.** Django 6.1 needs Python 3.12 or newer, and supports up to 3.15. Django 5.2 LTS supports Python 3.10 to 3.14. Flask 3.1 needs Python 3.9 or newer.
- **FastAPI** is another popular Python framework, built for JSON APIs. It's a good next step once you know Flask.

Setting up Python, virtual environments and project folders is covered in Python and Node Tool Environments. Before installing packages, read Safer Package Installs with npm and pip.

---

4. Node.js and Express: A Common Alternative

Node.js runs JavaScript outside the browser, so you can write the front end and back end in one language. **Express** is the classic, minimal web framework for Node.js, in the same spirit as Flask.

	Version (November 2026)	Notes
Node.js	**26 LTS** (Active LTS since October 28, 2026)	Node.js 24 and 22 are in maintenance. Production apps should use an LTS line
Express	**5** (5.3.0)	Express 5 needs Node.js 18 or newer. `npm install express` now installs version 5

What changed in Express 5, if you follow an older tutorial:

- **Errors in `async` routes are handled for you.** A rejected promise goes to your error handler, so you no longer need `try` / `catch` in every route just to pass errors on.
- **Route paths changed.** A wildcard needs a name (`/*` becomes `/*splat`), and optional parts use braces (`/:file{.:ext}`).
- **Some old methods are gone,** such as `req.param()` and `res.send(status)` . Use `req.params` , `req.query` or `req.body` , and `res.sendStatus()` .
- **`req.body` is `undefined`** when no body was parsed, rather than `{}` .

The Express team's migration guide lists every change, and offers a codemod that updates much of an old project for you.

Starting from JavaScript is a reasonable choice, especially if you've learned the front end first. Many job listings ask for Node.js, and full-stack frameworks such as Next.js run on it (Full-Stack Web Development and Stacks).

---

5. Languages to Learn Later

Once you're comfortable, these are worth learning, in order of how useful they're likely to be for back-end work. Again, there's no rush, and no need to learn them all.

Language	Where it's used on the back end
JavaScript (and TypeScript)	Node.js servers, Express APIs, serverless functions, and full-stack frameworks such as Next.js
Rust	Fast, memory-safe services and infrastructure, with web frameworks such as Axum and Actix Web
Go	Cloud services, APIs and infrastructure tools. Docker and Kubernetes are written in Go
C	The systems underneath: databases such as PostgreSQL and SQLite, and web servers such as nginx, rather than everyday app code
C++	High-performance systems such as database engines (MongoDB's server is C++), game servers and trading systems

Language	Where it's used on the back end
C#	ASP.NET Core web apps and APIs, common in companies that use Microsoft tools and Azure

To see how one program looks in three of these languages, watch the free preview of ThePrimeagen's course Polyglot Programming: TypeScript, Go, & Rust (Frontend Masters, now called Master.dev). The course builds the same tested command-line app in TypeScript, Go and Rust side by side, which makes the differences between the languages easy to see. The 17-minute preview gives a taste of three moments:

- **TypeScript:** unit tests for the course project, a small tool that stores settings per folder, where a folder inherits its parent folder's values unless it sets its own.
- **Go:** errors are ordinary return values, so you check `if err != nil` after nearly every call. He finds it repetitive but likes that it forces you to think about failure up front. Names starting with a capital letter are exported, and lower-case ones stay private.
- **Rust:** chaining iterators (`enumerate`, `filter`, `flat_map`, `count`) to solve a puzzle without building extra arrays, and handling `Option` values that may be empty.

In the preview he introduces himself as a Netflix engineer; he has since left Netflix. You can start the course for free on the site, and its pricing page covers the rest.

What about Carbon? Carbon is an experimental successor to C++, started by engineers at Google. As of November 2026, its own project page says it's **experimental and not ready for use**. It hasn't reached its first "0.1" milestone, a minimum version for people to try out, and there's no date for a 1.0 release. It's interesting to follow, but it isn't a job skill, so learn C++ if that's the area you want.

---

6. Free Courses and Roadmaps

- **CS50's Web Programming with Python and JavaScript** (Harvard, CS50W): free, with Python, Django, SQL and JavaScript. The full route is in Full-Stack Web Development and Stacks.
- **MDN: Server-side website programming**: a free introduction, then complete Django and Express tutorials.
- **The Odin Project: Full Stack JavaScript**: a free path that includes a Node.js course.
- **roadmap.sh: Backend Developer**: a map of back-end topics, to see what's ahead and what you can skip for now.
- **freeCodeCamp**: free courses that include databases and back-end material. Its older certifications are now "legacy", replaced by a new curriculum (including Relational Databases and Back End Development and APIs) that has been released in stages, so check the site for what's currently live.
- **Comparing curricula:** Start Here's table of free curricula puts The Odin Project, freeCodeCamp, MDN and CS50W side by side.
- **Official tutorials:** Django's tutorial, Flask's tutorial and Express's getting started guide.

---

7. HTTP, REST APIs and JSON

7.1 HTTP in one minute

Every request has a **method** (what to do), a **path** (what to do it to), **headers** (extra information, such as who's asking) and sometimes a **body** (the data you're sending). Every response has a **status code**, headers and usually a body.

Method	Means	Example
`GET`	Read something	`GET /api/notes`
`POST`	Create something	`POST /api/notes`
`PUT` or `PATCH`	Replace or change something	`PATCH /api/notes/7`
`DELETE`	Delete something	`DELETE /api/notes/7`

Status	Means
`200 OK`	It worked
`201 Created`	It worked, and something new was made
`204 No Content`	It worked, and there's nothing to send back
`400 Bad Request`	The request was wrong, such as a missing field
`401 Unauthorized`	You're not signed in
`403 Forbidden`	You're signed in, but not allowed to do this
`404 Not Found`	That thing doesn't exist
`500 Internal Server Error`	The server broke. That's a bug on your side

7.2 REST and JSON

A **REST API** organizes a back end around *things* (resources) with addresses, and uses the HTTP methods above as the verbs. `GET /api/notes` lists notes, and `DELETE /api/notes/7` deletes note 7.

JSON is the text format nearly every API uses to send data:

```
{ "id": 7, "text": "Buy milk", "done": false }
```

7.3 Build a tiny API in Flask

1. Set up a project folder and virtual environment (Python and Node Tool Environments explains each step), then install Flask:

   ```
   py -m venv .venv
   .venv\Scripts\Activate.ps1
   pip install flask
   ```

   On macOS or Linux, use `python3 -m venv .venv` and `source .venv/bin/activate`.
2. Save this as `app.py`:

```
from flask import Flask, request

app = Flask(__name__)
notes = []  # kept in memory, for practice only

@app.get("/api/notes")
def list_notes():
    return notes

@app.post("/api/notes")
def add_note():
    data = request.get_json(silent=True) or {}
    text = str(data.get("text", "")).strip()
    if not text:
        return {"error": "text is required"}, 400
    note = {"id": len(notes) + 1, "text": text}
    notes.append(note)
    return note, 201
```

Flask turns a returned `dict` or `list` into JSON for you.

3. Start it:

```
flask --app app run --debug
```

It runs at `http://127.0.0.1:5000`. Debug mode reloads when you save, and shows errors in the browser. Never use it on a real server.

7.4 The same API in Express

1. In a new folder:

```
npm init -y
npm install express
```

2. Save this as `server.js`:

```
const express = require("express");

const app = express();
app.use(express.json());
const notes = []; // kept in memory, for practice only

app.get("/api/notes", (req, res) => {
  res.json(notes);
});

app.post("/api/notes", (req, res) => {
  const text = String(req.body?.text ?? "").trim();
  if (!text) {
    return res.status(400).json({ error: "text is required" });
  }
  const note = { id: notes.length + 1, text };
  notes.push(note);
  res.status(201).json(note);
});

app.listen(3000, () => console.log("http://localhost:3000"));
```

3. Start it:

```
node server.js
```

7.5 Try it from the terminal

With either server running, open a second terminal. Use port `5000` for Flask, or `3000` for Express.

In PowerShell:

```
Invoke-RestMethod http://localhost:3000/api/notes
$body = @{ text = "Buy milk" } | ConvertTo-Json
Invoke-RestMethod http://localhost:3000/api/notes -Method Post `
  -ContentType "application/json" -Body $body
```

In Bash (macOS, Linux, WSL or Git Bash):

```
curl -i http://localhost:3000/api/notes
curl -i -X POST http://localhost:3000/api/notes \
  -H "Content-Type: application/json" -d '{"text":"Buy milk"}'
```

`-i` shows the status code and headers too. Try sending an empty `text` and check that you get `400`. Postman does the same in a window, if you prefer (Section 14).

---

8. Databases: SQL First, NoSQL When It Fits

The notes list above disappears every time the server restarts. A database keeps it.

8.1 Start with SQL

A **relational database** stores data in tables with rows and columns, and you talk to it in **SQL**. Learn this first: most jobs use it, and the ideas carry over everywhere.

```
CREATE TABLE notes (
  id INTEGER PRIMARY KEY,
  text TEXT NOT NULL,
  created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO notes (text) VALUES ('Buy milk');
SELECT id, text FROM notes WHERE id = 1;
```

Two good databases to learn on:

- **SQLite** keeps the whole database in one file, with no server to install. It's built into Python (`import sqlite3`), which makes it perfect for practice and small apps.
- **PostgreSQL** (version 18) is the free, full-featured database many companies and services run in production, including Supabase. Learn it next. Docker is an easy way to run it on your PC (Section 14).

Always pass user input to the database as **parameters**, never by pasting it into the SQL text. Pasting it in is how **SQL injection** happens:

```
import sqlite3

con = sqlite3.connect("notes.db")
con.execute("CREATE TABLE IF NOT EXISTS notes (id INTEGER PRIMARY KEY, text TEXT)")

text = "Buy milk"  # imagine this came from a user
con.execute("INSERT INTO notes (text) VALUES (?)", (text,))  # safe: a placeholder
con.commit()

# Never do this: con.execute(f"INSERT INTO notes (text) VALUES ('{text}')")
print(con.execute("SELECT id, text FROM notes").fetchall())
```

Once you're comfortable with SQL, most projects use an **ORM**, a library that maps tables to code: Django's built-in ORM, SQLAlchemy with Flask, or Prisma and Drizzle with Node.js. Learn plain SQL first, so you know what the ORM is doing for you.

8.2 When NoSQL fits

NoSQL databases store data in other shapes. The most common are **document databases**, such as MongoDB and Firebase's Cloud Firestore, which store JSON-like documents.

Choose	When
SQL (PostgreSQL, SQLite, MySQL)	Your data has clear relationships (users, orders, products), you need reliable transactions, or you're not sure. It's the safe default
Document database (MongoDB, Firestore)	Each record is a self-contained document that's read and written as a whole, its shape varies, or you want a managed service with real-time updates, as Firestore offers

Choose	When
Key-value store (such as Redis)	Caching, sessions, counters and queues, usually alongside a main database rather than instead of one

Many apps use more than one: SQL for the main data, plus a key-value store for caching.

Why are indexes and lookups fast or slow? Hash tables and trees, the structures behind most database indexes, are explained in Computer Science Fundamentals, Section 3. And if querying and summarizing data turns out to be the part you enjoy most, SQL is also the core of data analysis: see Learn Data Science, Step 2: SQL.

---

9. Authentication Basics

Authentication is checking *who* someone is (signing in). **Authorization** is checking *what they're allowed to do*. You need both, on every request.

1. **Don't build sign-in from scratch.** Use your framework's built-in system (Django has one), or a well-known library or service such as Better Auth (which now also maintains Auth.js), Firebase Authentication or Supabase Auth. Sign-in is easy to get subtly wrong.
2. **Never store passwords.** Store a slow, salted **hash** of each password. OWASP recommends **Argon2id**, then scrypt, with bcrypt for older systems. Django and good libraries handle this for you.
3. **Keep sessions in secure cookies.** After sign-in, the server gives the browser a session cookie. Mark it `HttpOnly` (scripts can't read it), `Secure` (HTTPS only) and `SameSite` (limits cross-site use).
4. **Know what a token is.** APIs often use tokens, such as JWTs, instead of cookies. They're convenient for mobile apps and services, but harder to cancel before they expire, so keep them short-lived.
5. **Check permission on the server, every time.** Hiding a button in the front end isn't security. If a user asks for `/api/notes/7`, check that note 7 is theirs. Skipping this is the number one risk on the OWASP Top 10.
6. **Offer more than a password.** Multi-factor authentication and passkeys stop most stolen-password attacks. "Sign in with Google" and similar options (OAuth) mean you never handle the password at all.
7. **Slow down guessing.** Limit how fast anyone can try to sign in, so attackers can't try thousands of passwords.

---

10. Web Security and the OWASP Top 10

10.1 The OWASP Top 10

The **OWASP Top 10** is the best-known list of the most serious web application risks. The current edition is **2025**:

ID	Risk	In plain words
A01	**Broken Access Control**	Users can see or change things that aren't theirs

ID	Risk	In plain words
A02	**Security Misconfiguration**	Debug mode on, default passwords, open storage buckets
A03	**Software Supply Chain Failures**	A package you install, or something it depends on, is malicious or compromised
A04	**Cryptographic Failures**	Data sent without HTTPS, weak or homemade encryption, plain-text passwords
A05	**Injection**	User input runs as code: SQL injection, command injection, cross-site scripting
A06	**Insecure Design**	The plan itself is unsafe, such as a password reset with no real check
A07	**Authentication Failures**	Weak sign-in: guessable passwords allowed, no rate limits, sessions that never expire
A08	**Software or Data Integrity Failures**	Trusting updates, files or data without checking where they came from
A09	**Security Logging and Alerting Failures**	Nobody notices an attack, because nothing was logged or nobody was alerted
A10	**Mishandling of Exceptional Conditions**	Errors that leak details, or leave the app in an unsafe state

The habits that prevent most of these:

- **Check access on the server** for every request (A01).
- **Use HTTPS everywhere,** and let your framework handle encryption and password hashing (A04).
- **Treat all input as untrusted:** validate it, use parameterized queries, and let your templates escape output (A05).
- **Turn debug mode off in production,** and show users plain error messages while logging the details privately (A02, A10).
- **Keep dependencies few and up to date,** and install carefully (A03). See Safer Package Installs with npm and pip.
- **Log sign-ins, failures and permission errors** somewhere you'll actually look (A09).

OWASP's free Cheat Sheet Series has a short, practical page for almost every topic here.

10.2 CORS, Origin and Host

Three ideas trip up nearly every back-end developer:

- **CORS** (Cross-Origin Resource Sharing) is a set of headers your server sends to tell **browsers** which other websites may read its responses. It's the reason a front end at one address gets a "CORS error" calling an API at another. Fix it by allowing your own front end's address, not by allowing `*` (everyone).
- **CORS isn't a lock on your server.** It only controls what a web page can *read*. Tools like `curl`, other servers and attackers' scripts ignore it, and a browser may still *send* some requests it won't let the page read. Your server still has to check every request.

- **The `Origin` and `Host` headers** tell your server which site a browser request came from, and which name it was sent to. Checking them is how a server refuses requests that come through tricks like DNS rebinding.

For a real-world example, read DNS Rebinding and the NVIDIA App's Local MCP Server, especially Why CORS Doesn't Stop It and How HTTP Header Validation Stops It.

---

11. Environment Variables and Secrets

Your code will need **secrets**: database passwords, API keys, and the secret key that signs sessions. They must never be written into your code or committed to Git. A good test: could you make your repository public right now without exposing anything?

1. **Keep settings in environment variables.** Your code reads them at startup, and each place it runs (your PC, the live server) sets its own values.

   ```
   import os

   SECRET_KEY = os.environ["SECRET_KEY"]  # stops with an error if it's missing
   ```

   ```
   const apiKey = process.env.API_KEY;
   if (!apiKey) throw new Error("API_KEY is not set");
   ```

2. **On your PC, use a `.env` file.** List one `NAME=value` per line, and load it with `python-dotenv` in Python, or Node.js's built-in `--env-file` option:

   ```
   node --env-file=.env server.js
   ```

3. **Keep `.env` out of Git.** Add it to `.gitignore` *before* your first commit. Commit a `.env.example` with the names and fake values instead, so others know what to set. The custom-domain guide has an example `.gitignore` for secrets.
4. **Make strong secrets with a random generator,** such as:

   ```
   python -c "import secrets; print(secrets.token_hex())"
   ```

5. **In production, set secrets in your host's settings or secret manager,** never in a file you upload. Firebase App Hosting, for example, reads them from Cloud Secret Manager (Section 4.5 of the custom-domain guide).
6. **If a secret leaks, replace it.** Deleting the commit isn't enough, because copies of the history may already exist. Create a new key, revoke the old one, then clean up. GitHub's secret scanning can warn you about leaked keys in public repositories.

---

12. Testing

Tests are small programs that check your code still does what it should. They let you change things without fear, and employers look for them in portfolio projects.

Kind	Checks	Example
Unit test	One function on its own	"Does `slugify("Hi There")` return `hi-there`?"
Integration test	Several pieces together, such as a route and the database	"Does `POST /api/notes` save a note and answer `201`?"
End-to-end test	The whole app, through a real browser	"Can a user sign up, add a note and see it?"

Write mostly unit and integration tests, and a few end-to-end tests for the most important journeys.

Tools to know:

- **Python: pytest** is the most popular test runner. Flask has a built-in test client, and Django has its own test framework (`python manage.py test`).
- **Node.js:** the built-in test runner (`node --test`), or Vitest or Jest.

A test for the Flask API from Section 7, saved as `test_app.py` next to `app.py`:

```
from app import app

def test_add_note():
    client = app.test_client()
    res = client.post("/api/notes", json={"text": "Buy milk"})
    assert res.status_code == 201
    assert res.get_json()["text"] == "Buy milk"

def test_rejects_empty_note():
    client = app.test_client()
    res = client.post("/api/notes", json={"text": ""})
    assert res.status_code == 400
```

Run it:

```
pip install pytest
pytest
```

Once you have tests, run them automatically on every push with GitHub Actions. Section 10 of Building Firebase Apps in Google Antigravity IDE shows a real setup.

---

13. Deployment Basics

Deploying means running your back end on a server that's always on and reachable from the internet. The main options:

Option	What it is	Good for
Platform as a service	You push your code, and the platform builds, runs and scales it. Examples include Firebase App Hosting, Render, Railway and Fly.io	Most learners and small projects
Containers	You package your app with Docker, then run the container on a service such as Google Cloud Run	When you want the same setup everywhere
Virtual private server (VPS)	A rented Linux machine you manage yourself	Learning how servers work, at the cost of doing all the upkeep

Free tiers change often, so check each platform's pricing page, and set a budget alert where you can.

Before you go live:

1. **Use a production server, not the development one.** Flask's docs list production servers such as Gunicorn and Waitress. For Django, run `python manage.py check --deploy` and work through its warnings.
2. **Turn debug mode off.**
3. **Set your environment variables and secrets** on the host (Section 11).
4. **Use HTTPS.** Most platforms do it for you.
5. **Host the database too,** back it up, and run your migrations (database changes) as part of deploying.
6. **Read your logs** after each release, so you see errors before users report them.
7. **Point your own domain at it,** if you have one.

For a complete real example, from DNS to deploying updates, see Connecting a Custom Domain to Firebase App Hosting. Its Section 9 is a good look at how the `Host` header behaves once your app sits behind a hosting platform's proxy. To test and deploy automatically on every push, see CI/CD with GitHub Actions in the DevOps guide.

---

14. Tools: Charm, Postman, curl and Docker

14.1 Charm

Charm (the address now leads to charm.land) makes free, open-source tools for building good-looking terminal apps, mostly in Go. Supabase's four-minute video Open Source Spotlight: Charm.sh is the Charm team introducing themselves: their aim is to make the command line glamorous, treating software as part art and part engineering. They walk through their libraries (Bubble Tea, Bubbles, Lip Gloss, Glamour, Harmonica and Wish) and their apps (Glow, Soft Serve, Melt, Wishlist and Skate), plus Charm Cloud. Several of those have changed since (see the note below the table).

What Charm offers as of November 2026:

Tool	What it does
Bubble Tea	A framework for building terminal user interfaces (TUIs), with **Bubbles** as ready-made components

Tool	What it does
Lip Gloss	Styling and layout for terminal output
Huh	Forms and prompts in the terminal
Wish	Building apps people reach over SSH, so a terminal app can be used from anywhere without installing it
Gum	Polished shell scripts (prompts, menus, spinners) without writing Go
Glow and **Glamour**	Reading and rendering Markdown in the terminal
VHS	Scripting terminal recordings as GIFs, handy for READMEs
Log	A small, friendly logger
Skate	A personal key-value store, now stored locally
Pop	Sending email from the terminal, made with Resend
Crush	An AI coding assistant for the terminal, which works with the AI model you choose
Harmonica	A small, physics-based animation library, for springy motion in terminal or game interfaces
Melt	Backing up an Ed25519 SSH private key as a list of seed words you can write down, and restoring it from them
Wishlist	A directory of SSH apps and servers, so one entry point lists them all

Charm's site also lists **Hyper**, an inference platform for coding work.

> **Note:** The Supabase video, and older articles, also describe **Charm Cloud**: hosted encryption keys, a file system and syncing, which Glow's "stash" and Skate used. Charm shut Charm Cloud down on **November 29, 2024**. Its code is still open source if you want to host it yourself, and Skate and Glow no longer use it. The video also shows **Soft Serve**, a self-hosted Git server you browse over SSH. Charm has since sunset it: its repository was archived in October 2026, and it no longer gets security updates, so don't start new projects on it.

For back-end developers, Bubble Tea and Gum are great for admin tools and scripts, and Wish lets you offer a terminal app over SSH.

14.2 Postman and curl

You'll test APIs constantly. Two ways:

- `curl` sends a request from the terminal, as in Section 7.5. It's built into Windows 10 and 11, macOS and Linux. In Windows PowerShell 5.1, type `curl.exe`, because `curl` there is a different command. The CLI Cheat Sheet's networking section has more `curl` examples.
- **Postman** is an app for building, saving and repeating API requests in a window. Its free plan covers one user, which is plenty for learning.

14.3 Docker

Docker packages an app with everything it needs into a **container**, so it runs the same on any machine. It's also the easiest way to run a database on your PC without installing it.

1. Install Docker Desktop. On Windows, it runs on WSL 2. The CLI Cheat Sheet's developer setup section has the install commands. Docker Desktop is free for personal use, education, non-commercial open source, and businesses with fewer than 250 employees *and* under $10 million in yearly revenue.
2. Start PostgreSQL 18 in a container, with your own local password in place of `<a-local-password>`:

   ```
   docker run --name dev-postgres -e POSTGRES_PASSWORD=<a-local-password> `
     -p 5432:5432 -v pgdata:/var/lib/postgresql -d postgres:18
   ```

   `-p` makes it reachable at `localhost:5432`, and `-v` keeps the data in a volume called `pgdata`, so it survives the container being removed. (PostgreSQL 18 images store data under `/var/lib/postgresql`. For version 17 and older, use `/var/lib/postgresql/data`.)
3. Stop it with `docker stop dev-postgres`, and start it again with `docker start dev-postgres`.

The CLI Cheat Sheet's Docker section covers containers, images, logs, building your own images, and Docker Compose for running an app and its database together. For containers as part of a build-and-deploy pipeline, see Getting Started with DevOps, Section 9.

---

15. Common Mistakes

Mistake	Why it hurts	Do this instead
Committing `.env` or an API key	Anyone who sees the repository can use it, and it stays in the history	Add `.gitignore` first. If it happens, replace the key (Section 11)
Storing passwords as plain text, or with a fast hash such as SHA-256	One database leak exposes every user	Use your framework's sign-in, or Argon2id (Section 9)
Building SQL by pasting in user input	SQL injection	Use parameters or an ORM (Section 8)
Only hiding buttons in the front end	Anyone can call your API directly	Check permissions on the server (Section 9)
Setting CORS to allow `*` to "fix" an error	Any website can read your API's responses	Allow only your own front end's address (Section 10.2)
Running `flask run --debug` or Django's `runserver` in production	Debug pages can expose your code and settings	Use a production server, debug off (Section 13)
Learning three frameworks at once	Slow progress, nothing finished	One language, one framework, real projects (Section 2)

---

16. Sources

Checked against these pages in November 2026.

Languages and frameworks

- Python downloads and release status
- OpenJDK: JDK releases and Oracle Java downloads (JDK 25 LTS)
- Spring Boot
- Django download and supported versions and Django's Python version support
- Flask on PyPI and Flask quickstart
- FastAPI
- Node.js previous releases and Node.js release schedule
- Express and Migrating to Express 5
- Rust, Go, ASP.NET Core
- Carbon Language on GitHub

Courses

- CS50's Web Programming with Python and JavaScript
- MDN: Server-side website programming
- The Odin Project: Full Stack JavaScript
- roadmap.sh: Backend Developer
- freeCodeCamp

HTTP, databases and security

- MDN: Overview of HTTP and HTTP response status codes
- MDN: Cross-Origin Resource Sharing (CORS) and Using HTTP cookies
- PostgreSQL, SQLite and Python's sqlite3 module
- OWASP Top 10:2025
- OWASP Password Storage Cheat Sheet
- Better Auth and Auth.js
- The Twelve-Factor App: Config
- GitHub: About secret scanning

Testing

- pytest
- Node.js test runner

Tools

- Charm and Sunsetting Charm Cloud
- Charm on GitHub: Soft Serve (archived), Melt, Wishlist and Harmonica
- curl
- Postman pricing
- Docker: Get started and Docker Desktop license agreement

- The official PostgreSQL Docker image

Videos from the original notes (each video's content checked in November 2026)

- Aaron Jack, Learning Back End? You NEED to know these topics.
- Supabase, Open Source Spotlight: Charm.sh
- Frontend Masters (Master.dev), Polyglot Programming: TypeScript, Go, & Rust by ThePrimeagen | Preview, and the full course page

---

Latest version: github.com/ImmuneMoon/Learn-Back-End-Web-Development

If this helped, you can support Fulllion Creative Works through PayPal or Ko-fi.

© 2026 Fulllion Creative Works. All rights reserved.